HELLA company 업무관리사이트 — 개발 직전 통합 구현 설계서

이 문서는 다음 4개를 하나로 통합한 **실행 직전 기준 캔버스**다.

1. 필드 단위 화이트리스트 기반 Firestore Rules 강화안
2. 상태 전이 검증 로직 완전 봉쇄 설계
3. Firebase Functions 설계 (NFC / 장비 / 로그 / 일정 상태 동기화)
4. 실제 프로젝트 코드 구조 (React + Firebase)

이 문서는 기존 MVP 설계, 메뉴 기능 명세, 프로젝트 로드맵, Screen Map, 로그인/사원/관리자/NFC/Rules 와이어프레임을 모두 반영한 **개발 SSOT 보조 문서**로 사용한다.

---

0. 전제

0-1. 현재 프로젝트 상태

이 프로젝트는 아이디어 단계가 아니라 **개발 직전 단계**다.

이미 확정된 것: - 역할 구조 확정 - 메뉴 구조 확정 - 주요 컬렉션 구조 확정 - 관리자/사원 주요 화면 와이어프레임 확정 - NFC 반출/반입 엔진 설계 확정 - Firestore Rules v1 방향 확정

즉, 지금 필요한 것은 추가 기획이 아니라 **보안 강화 + 상태 전이 통제 + 서버 함수 구조 + 실제 코드 베이스 구조 고정**이다.

0-2. 반드시 유지할 핵심 철학

1. 상태 기반 설계
2. 트랜잭션 기반 처리
3. UI와 비즈니스 로직 분리
4. 서버 기준 상태 판정
5. 모든 중요 액션 로그화
6. 사원은 입력 중심, 상태 확정은 서버 중심
7. 확장 가능한 구조 유지

0-3. 역할 고정

- EMPLOYEE
- LEADER
- ADMIN
- SUPER_ADMIN

0-4. 핵심 컬렉션 고정

- users

- business_categories
- business_subcategories
- manuals
- schedules
- schedule_workers
- schedule_closings
- schedule_photos
- regular_cleaning_sites
- regular_cleaning_attendance
- equipments
- equipment_logs
- nfc_tag_mappings
- audit_logs
- download_logs
- settings

---

1. 필드 단위 화이트리스트 — 초정밀 Rules 강화 설계

이 장의 목적은 단순히 컬렉션 접근을 막는 수준이 아니라, **각 문서에서 누가 어떤 필드만 바꿀 수 있는지**를 정확히 고정하는 것이다.

핵심 원칙: - 읽기 권한과 쓰기 권한은 별개로 본다. - update 허용은 곧 전체 문서 자유 수정이 아니다. - 본인 문서 수정 허용 시에도 수정 가능한 필드를 화이트리스트로 제한한다. - 서버 함수 전용 컬렉션은 클라이언트 create/update/delete를 모두 차단한다.

1-1. users

문서 목적

사원 프로필, 계정 상태, 역할, 부서, 직급, 인증 후 권한 판정의 기준

권한 원칙

- 본인: 읽기 가능
- ADMIN 이상: 전체 읽기 가능
- 생성: ADMIN 이상 또는 Functions 전용 권장
- 삭제: 금지

본인 수정 가능 필드

- displayName
- phone
- profileImageUrl
- emergencyContact (필요 시)
- updatedAt

본인 수정 금지 필드

- uid
- role
- isActive
- employmentStatus
- department
- position
- joinedAt
- monthlyStats
- createdAt
- lastLoginAt
- permissionFlags

관리자 수정 가능 필드

- displayName
- phone
- profileImageUrl
- role
- isActive
- employmentStatus
- department
- position
- joinedAt
- managerId
- updatedAt

관리자도 직접 수정하지 않는 필드

- createdAt
- authProviderUid 불변 필드
- 집계 캐시 필드 일괄 재계산 값

권장 화이트리스트 함수 개념

- hasOnlyAllowedKeysForSelfUserUpdate()
- hasOnlyAllowedKeysForAdminUserUpdate()

1-2. business_categories

관리자 수정 가능 필드

- name
- description
- sortOrder
- isActive
- updatedAt

생성 시 필수 필드

- name
- isActive
- sortOrder
- createdAt
- updatedAt

금지

- 물리 삭제
- 생성 후 시스템 내부 식별자 변경

1-3. business_subcategories

관리자 수정 가능 필드

- categoryId
- name
- description
- sortOrder
- isActive
- allowCustomInput
- updatedAt

금지

- 삭제
- 상위 categoryId 무단 교체 후 기존 일정과 정합성 깨지는 변경

1-4. manuals

관리자 수정 가능 필드

- categoryId
- subcategoryId
- title
- content
- version
- isLatest
- isActive
- updatedAt
- thumbnailUrl

생성 시 필수 필드

- title
- content
- version

- authorUid
- createdAt
- updatedAt
- isActive

사원 수정

- 전부 금지

금지

- createdAt 변경
- authorUid 위조
- 이전 버전 문서 덮어쓰기

1-5. schedules

목적

일정 마스터 문서

읽기

- ADMIN 이상 전체
- EMPLOYEE/LEADER는 본인 배정된 일정만

관리자 생성 가능 필드

- scheduleTitle
- categoryId
- subcategoryId
- siteAddress
- siteAddressDetail
- scheduleDate
- startTime
- expectedEndTime
- workerIds
- leaderUserId
- customerName
- customerPhone
- specialNotes
- extraMemo
- status
- assignedWorkerNamesCache
- hasClosingRecord
- hasBeforePhotos
- hasAfterPhotos
- createdBy
- createdAt
- updatedAt

- cancelledAt
- cancelledReason

관리자 수정 가능 필드

- scheduleTitle
- categoryId
- subcategoryId
- siteAddress
- siteAddressDetail
- scheduleDate
- startTime
- expectedEndTime
- workerIds
- leaderUserId
- customerName
- customerPhone
- specialNotes
- extraMemo
- status
- assignedWorkerNamesCache
- hasClosingRecord
- hasBeforePhotos
- hasAfterPhotos
- updatedAt
- cancelledAt
- cancelledReason

사원 직접 수정 금지 필드

- schedules 전체 원칙적 직접 수정 금지

서버 함수 전용 필드 권장

- status
- hasClosingRecord
- hasBeforePhotos
- hasAfterPhotos
- delayedAt
- completedAt

이유: 일정 상태는 개별 사원 입력 결과를 종합해 판정해야 하므로 클라이언트 직접 변경 허용 시 무결성이 깨진다.

1-6. schedule_workers

목적

사원별 일정 참여 상태 문서

관리자 생성 가능 필드

- scheduleId
- workerId
- workerName
- assignedRole
- assignmentStatus
- workStatus
- actualStartAt
- actualEndAt
- participationApproved
- adminNote
- createdAt
- updatedAt
- assignedBy

사원 본인 수정 가능 필드

- actualStartAt
- actualEndAt
- updatedAt

사원 본인 조건부 수정 가능 필드

- workStatus
- 서버 함수 호출 결과로만 허용 권장
- 클라이언트 직접 update 허용 시 상태 전이 검증 필수

사원 본인 수정 금지 필드

- scheduleId
- workerId
- workerName
- assignedRole
- assignmentStatus
- participationApproved
- adminNote
- assignedBy
- createdAt

관리자 수정 가능 필드

- assignmentStatus
- workStatus
- participationApproved
- adminNote
- assignedRole
- updatedAt

1-7. schedule_closings

목적

사원 현장 마감 공식 기록

사원 생성 가능 필드

- scheduleId
- workerId
- startAt
- endAt
- notes
- submittedAt
- status
- beforePhotoCount
- afterPhotoCount
- createdAt
- updatedAt

사원 생성 시 금지 필드

- approvedBy
- reviewStatus
- adminReviewNote
- forceAdjustedBy
- finalizedAt

사원 수정 가능 여부

- 원칙: 생성 후 수정 금지 또는 draft 상태에서 notes 정도만 제한 허용
- 권장: 최종 제출 문서는 사원 수정 금지, 임시저장용 별도 draft 전략 또는 notes만 제출 전 수정 허용

관리자 수정 가능 필드

- reviewStatus
- adminReviewNote
- approvedBy
- approvedAt
- forceAdjustedBy
- forceAdjustedReason
- updatedAt

삭제

- 금지

1-8. schedule_photos

목적

비포/애프터 사진 메타데이터

생성 가능 필드

- scheduleId
- uploaderId
- photoType
- title
- storagePath
- fileName
- fileSize
- uploadedAt
- createdAt

업로더 본인 수정 가능 필드

- title
- updatedAt

업로더 본인 수정 금지 필드

- scheduleId
- uploaderId
- photoType
- storagePath
- fileName
- fileSize
- uploadedAt
- createdAt

관리자 수정 가능 필드

- title
- moderationStatus
- updatedAt

삭제

- 업로더 본인 또는 관리자 가능
- 단, 운영 안정성 위해 soft delete 고려 가능

1-9. regular_cleaning_sites

관리자 수정 가능 필드

- name

- address
- defaultNotes
- isActive
- assignedDefaultWorkerIds
- updatedAt

삭제

- 금지

1-10. regular_cleaning_attendance

관리자 생성 가능 필드

- siteId
- dateKey
- workerId
- workerName
- startAt
- endAt
- participationApproved
- createdAt
- updatedAt

사원 본인 수정 가능 필드

- startAt
- endAt
- updatedAt

사원 수정 금지 필드

- siteId
- dateKey
- workerId
- workerName
- participationApproved
- createdAt

관리자 수정 가능 필드

- participationApproved
- adminAdjustmentNote
- updatedAt

1-11. equipments

목적

장비 현재 상태의 기준 문서

관리자 생성 가능 필드

- equipmentCode
- name
- serialNumber
- category
- storageLocation
- note
- isActive
- status
- nfcTagId
- currentHolderUserId
- currentHolderName
- lastCheckoutAt
- lastCheckinAt
- createdAt
- updatedAt

관리자 수동 수정 가능 필드

- equipmentCode
- name
- serialNumber
- category
- storageLocation
- note
- isActive
- updatedAt

관리자도 직접 수정 자제해야 하는 상태 필드

- status
- currentHolderUserId
- currentHolderName
- lastCheckoutAt
- lastCheckinAt

이 필드들은 반출/반입 트랜잭션에서만 변경해야 한다.

일반 사원 수정

- 전부 금지

1-12. nfc_tag_mappings

관리자 생성 가능 필드

- nfcTagId
- equipmentId
- status
- mappedAt
- mappedBy
- createdAt
- updatedAt

관리자 수정 가능 필드

- equipmentId
- status
- updatedAt

일반 사원 접근

- 읽기/쓰기 모두 금지 권장

삭제

- 금지

1-13. equipment_logs

원칙

클라이언트 create/update/delete 전부 금지

서버 함수 생성 필드

- equipmentId
- equipmentCode
- equipmentName
- nfcTagId
- actionType
- actorUserId
- actorUserName
- reason
- useLocation
- note
- result
- performedAt
- previousStatus
- nextStatus
- currentHolderUserId

수정/삭제

- 전부 금지

1-14. audit_logs

원칙

감사 로그는 증거 데이터다.

생성

- Functions 전용 권장

허용 필드

- actorUserId
- actorUserName
- actorRole
- targetCollection
- targetDocumentId
- action
- reason
- beforeData
- afterData
- createdAt

수정/삭제

- 전부 금지

1-15. download_logs

n

생성

- 관리자 또는 Functions

수정/삭제

- 금지

필드 예시

- actorUserId
- actorUserName
- exportType
- filterSnapshot

- createdAt

1-16. settings

관리자 수정 가능 필드

- companyName
- companyPhone
- companyAddress
- supportEmail
- loginLogoUrl
- headerLogoUrl
- defaultNewUserRole
- photoPolicy
- nfcPolicy
- updatedAt

SUPER_ADMIN 전용 권장 필드

- systemFlags
- rolePolicy
- emergencyLockdown
- hiddenFeatureToggles

일반 사원 접근

- 금지

1-17. Rules Helper 설계 방향

Rules는 아래 helper 집합으로 강화한다.

인증/역할 helper

- isSignedIn()
- getCurrentUser()
- isActiveUser()
- role()
- isLeader()
- isAdmin()
- isSuperAdmin()
- isSelf(uid)

필드 화이트리스트 helper

- hasOnlyKeys(allowedKeys)
- unchanged(field)
- requiredOnCreate(field)
- isValidEnum(field, allowedValues)

상태 비교 helper

- prev(field)
- next(field)
- changedOnly(allowedKeys)

시간/제약 helper

- isServerManagedFieldUnchanged()
- isNotBackdatingDangerousField()

---

2. 상태 전이 검증 로직 — 완전 봉쇄 설계

이 장의 목적은 단순 상태 목록이 아니라, **어떤 상태에서 어떤 상태로만 이동 가능한지**, 그리고 **누가 그 전이를 발생시킬 수 있는지**를 고정하는 것이다.

핵심 원칙: - 상태는 문자열이 아니라 규칙이다. - 허용 목록에 없는 전이는 모두 차단한다. - 사원은 상태를 “선언”하는 것이 아니라 입력 행위를 남긴다. - 최종 상태는 서버가 계산한다.

2-1. schedules 상태 머신

상태 enum

- PLANNED
- IN_PROGRESS
- COMPLETED
- CANCELLED
- DELAYED

생성 시 기본값

- PLANNED

허용 전이

- PLANNED → IN_PROGRESS
- IN_PROGRESS → COMPLETED
- PLANNED → CANCELLED
- IN_PROGRESS → DELAYED
- DELAYED → COMPLETED
- DELAYED → CANCELLED

제한 전이

- COMPLETED → IN_PROGRESS 금지
- CANCELLED → PLANNED 금지
- CANCELLED → IN_PROGRESS 금지
- COMPLETED → PLANNED 금지

누가 전이를 일으키는가

- PLANNED → IN_PROGRESS
- 서버 함수가 schedule_workers 시작 입력을 보고 계산
- IN_PROGRESS → COMPLETED
- 서버 함수가 모든 필수 마감 조건 충족 시 계산
- PLANNED → CANCELLED
- ADMIN 이상
- IN_PROGRESS → DELAYED
- 서버 함수 또는 관리자 정책 처리

서버 판정 기준 예시

- 하나 이상의 assigned worker가 actualStartAt 입력 → IN_PROGRESS 후보
- 전체 필수 worker가 종료 또는 closing 제출 완료 → COMPLETED 후보
- expectedEndTime 초과 + closing 미제출 → DELAYED 후보

2-2. schedule_workers 상태 머신

상태 enum

- ASSIGNED
- STARTED
- ENDED
- CLOSED
- REVIEW_REQUIRED
- CANCELLED

생성 시 기본값

- ASSIGNED

사원 허용 전이

- ASSIGNED → STARTED
- STARTED → ENDED

관리자 허용 전이

- ENDED → CLOSED
- ANY → REVIEW_REQUIRED
- ASSIGNED → CANCELLED
- STARTED → CANCELLED (강제 처리 필요 시)

금지 전이

- ASSIGNED → ENDED 직접 금지
- STARTED → ASSIGNED 금지
- ENDED → STARTED 금지
- CLOSED → STARTED 금지
- CLOSED → ENDED 금지

필수 검증

- STARTED 전 actualStartAt 존재 필수
- ENDED 전 actualEndAt 존재 필수
- actualEndAt > actualStartAt 필수
- CLOSED 전 closing record 존재 여부 또는 관리자 강제사유 필요

권장 구현 방식

클라이언트가 workStatus를 직접 set하지 말고: - recordWorkStart() - recordWorkEnd() - submitClosing() 이 3개 서버 함수가 상태를 변경하게 한다.

2-3. schedule_closings 상태 머신

상태 enum

- DRAFT
- SUBMITTED
- REVIEWED
- APPROVED
- REJECTED
- FORCE_ADJUSTED

사원 허용 전이

- 없음 또는 DRAFT 생성
- DRAFT → SUBMITTED

관리자 허용 전이

- SUBMITTED → REVIEWED
- REVIEWED → APPROVED
- REVIEWED → REJECTED
- APPROVED → FORCE_ADJUSTED
- REJECTED → FORCE_ADJUSTED

필수 검증

- SUBMITTED 전 startAt 존재
- SUBMITTED 전 endAt 존재
- SUBMITTED 전 endAt > startAt
- 정책상 최소 사진 수 조건 충족 여부 검사
- 업로드 실패 사진 남아 있으면 SUBMITTED 차단

금지 전이

- APPROVED 이후 사원 수정 금지
- SUBMITTED → DRAFT 복귀 금지
- REJECTED 문서 임의 삭제 금지

2-4. equipments 상태 머신

상태 enum

- AVAILABLE
- CHECKED_OUT
- OVERDUE
- DISABLED

허용 전이

- AVAILABLE → CHECKED_OUT
- CHECKED_OUT → AVAILABLE
- CHECKED_OUT → OVERDUE
- OVERDUE → AVAILABLE
- AVAILABLE → DISABLED
- CHECKED_OUT → DISABLED (관리자 예외)
- OVERDUE → DISABLED (관리자 예외)
- DISABLED → AVAILABLE (관리자 복구)

일반 사용자 금지 전이

- 상태 직접 변경 전부 금지

서버 함수 전용 전이

- checkoutEquipment()
- checkinEquipment()
- markEquipmentOverdue()
- forceUpdateEquipmentStatus()

검증 규칙

- CHECKED_OUT 전 equipment.isActive = true
- CHECKED_OUT 전 nfc_tag_mapping ACTIVE 존재
- CHECKED_OUT 전 status = AVAILABLE
- AVAILABLE 복귀 전 status ∈ {CHECKED_OUT, OVERDUE}
- DISABLED 상태 일반 처리 금지

2-5. regular_cleaning_attendance 상태

상태 또는 파생 상태

- ASSIGNED
- STARTED
- ENDED
- APPROVED
- MISSED

규칙

- startAt 입력 전 ENDED 금지
- endAt 입력 후 관리자 승인 시 APPROVED
- 배정만 있고 입력 없으면 MISSED 판정 가능

2-6. 전이 검증 우선순위

상태 전이 검증은 아래 순서로 본다.

1. 사용자 권한 검증
2. 현재 문서 존재 및 활성 여부 검증
3. 이전 상태 검증
4. 입력 필수값 존재 여부 검증
5. 시간 역전 여부 검증
6. 관련 문서 정합성 검증
7. 전이 허용 여부 검증
8. 트랜잭션 커밋
9. 로그 생성

2-7. 일정 상태 계산 규칙 권장안

schedule.status 계산은 직접 입력 금지

서버가 계산한다.

계산 예시

- any(schedule_workers.actualStartAt != null) 이면 IN_PROGRESS 후보
- all(required workers workStatus in [ENDED, CLOSED]) + closing 존재 시 COMPLETED 후보
- expectedEndTime 초과 + 미제출 일정 존재 시 DELAYED 후보
- 관리자 취소 시 CANCELLED

캐시 필드 허용

프론트 조회 성능을 위해 schedules에 아래 캐시 필드를 둘 수 있다. - activeWorkerCount - startedWorkerCount - endedWorkerCount - closingSubmittedCount - needsClosingCount

단, 이 값은 모두 서버 함수만 갱신한다.

---

3. Firebase Functions 설계 — NFC / 장비 / 로그 / 트랜잭션

이 장의 목적은 클라이언트가 해서는 안 되는 작업을 서버 함수로 고정하는 것이다.

핵심 원칙: - 중요 상태 변경은 Functions 전용 - 하나의 상태 전이 = 하나의 트랜잭션 단위 - 상태 변경 시 로그 동시 생성 - 교차 컬렉션 정합성은 함수에서 책임진다

권장 리전: - asia-northeast3

권장 방식: - Callable Functions 우선 - 일부 배치/스케줄러 함수 병행

3-1. Functions 책임 범위

인증/권한 관련

- 관리자 계정 생성
- 역할 부여/변경 보조

일정 관련

- createScheduleWithWorkers
- updateScheduleAndWorkers
- cancelSchedule
- recomputeScheduleStatus
- submitClosing
- approveClosing

장비 관련

- assignNfcTag
- checkoutEquipment
- checkinEquipment
- forceUpdateEquipmentStatus
- markEquipmentOverdueBatch

로그 관련

- appendAuditLog
- createDownloadLog

집계/동기화 관련

- syncSchedulePhotoFlags
- recomputeDashboardMetrics

3-2. 함수 목록 — 일정 도메인

createScheduleWithWorkers(payload)

목적

일정 마스터 생성 + 배정 레코드 동시 생성

입력

- scheduleTitle
- categoryId
- subcategoryId
- siteAddress
- siteAddressDetail
- scheduleDate
- startTime
- expectedEndTime
- workerIds[]
- leaderUserId
- customerName
- customerPhone
- specialNotes
- extraMemo

처리

1. 호출자 ADMIN 이상 검증
2. 입력값 검증
3. 비활성 사원 포함 여부 검증
4. schedules 문서 생성
5. workerIds 수만큼 schedule_workers 생성
6. assignedWorkerNamesCache 계산
7. audit_logs 생성
8. 트랜잭션 커밋

출력

- scheduleId
- createdWorkerCount

---

updateScheduleAndWorkers(payload)

목적

일정 수정 + 배정 변경 + 캐시 갱신

처리 주의

- 이미 STARTED 상태인 사원 제거 시 경고 또는 차단 정책 필요
- category/subcategory 변경 시 메뉴얼 연계 영향 검토

처리

1. ADMIN 이상 검증
2. 기존 schedule 조회
3. 현재 schedule_workers 조회
4. 변경 diff 계산
5. 허용 가능한 변경인지 검증

6. schedule update
7. worker add/remove/update
8. assignedWorkerNamesCache 재계산
9. audit_logs 생성

---

cancelSchedule(payload)

목적

일정 취소

입력

- scheduleId
- reason

처리

1. ADMIN 이상 검증
2. 현재 상태 조회
3. COMPLETED 일정 취소 정책 확인
4. schedules.status = CANCELLED
5. schedule_workers.assignmentStatus 또는 workStatus 보조 처리
6. audit_logs 생성

---

recordWorkStart(payload)

목적

사원의 실제 시작시간 기록

입력

- scheduleId
- startAt(optional, 미입력 시 server now)

처리

1. 호출자 본인 배정 일정인지 검증
2. schedule_workers 문서 조회
3. 현재 workStatus = ASSIGNED 인지 검증
4. actualStartAt 기록
5. workStatus = STARTED
6. schedules 상태 재계산
7. audit 또는 activity log 생성 선택

---

recordWorkEnd(payload)

목적

사원의 실제 종료시간 기록

처리

1. 본인 배정 일정 검증
2. current workStatus = STARTED 검증
3. actualEndAt > actualStartAt 검증
4. actualEndAt 기록
5. workStatus = ENDED
6. schedules 상태 재계산

---

submitClosing(payload)

목적

최종 마감 제출

입력

- scheduleId
- notes
- beforePhotoIds[]
- afterPhotoIds[]
- startAt
- endAt

처리

1. 본인 배정 일정 검증
2. start/end/사진 정책 검증
3. schedule_closings upsert 또는 create
4. status = SUBMITTED
5. schedule_workers.workStatus = CLOSED 또는 CLOSING_SUBMITTED 정책 반영
6. schedules.hasClosingRecord = true
7. schedules.hasBeforePhotos / hasAfterPhotos 갱신
8. schedules 상태 재계산
9. audit/log 생성

---

approveClosing(payload)

목적

관리자 마감 승인

처리

1. ADMIN 이상 검증
2. closing 조회
3. reviewStatus / approve fields 기록
4. 필요한 경우 schedule_workers.workStatus = CLOSED 확정
5. schedules 상태 재계산
6. audit_logs 생성

3-3. 함수 목록 — 장비/NFC 도메인

assignNfcTag(payload)

목적

NFC 태그와 장비 1:1 매핑

입력

- equipmentId
- nfcTagId

처리

1. ADMIN 이상 검증
2. nfcTagId 정규화
3. 기존 ACTIVE 매핑 중복 확인
4. 장비 존재 및 활성 확인
5. nfc_tag_mappings 생성 또는 기존 비활성 후 신규 생성
6. equipments.nfcTagId 반영
7. equipment_logs 또는 audit_logs 기록

---

processNfcScan(payload)

목적

스캔 후 허용 액션만 판정

입력

- nfcTagId
- mode (CHECK_OUT | CHECK_IN)

처리

1. 사용자 활성 검증
2. mapping 조회
3. equipment 조회
4. equipment.status 확인

5. mode 적합성 판정
6. allowedAction / message 반환

비고

이 함수는 상태를 바꾸지 않는다. 판단 전용이다.

---

checkoutEquipment(payload)

목적

장비 반출 트랜잭션

입력

- nfcTagId
- equipmentId(optional)
- reason
- useLocation
- note

트랜잭션 처리

1. actor ACTIVE 검증
2. nfc mapping 조회
3. equipment 최신 상태 재조회
4. status == AVAILABLE 인지 검증
5. equipments.status = CHECKED_OUT
6. currentHolderUserId/name 설정
7. lastCheckoutAt = serverTimestamp
8. equipment_logs CHECK_OUT 생성
9. audit_logs 필요 시 생성
10. 커밋

실패 케이스

- 이미 반출 중
- 비활성 장비
- 태그 미등록
- 태그 매핑 불일치
- 동시성 충돌

---

checkinEquipment(payload)

목적

장비 반입 트랜잭션

입력

- nfcTagId
- note

트랜잭션 처리

1. actor ACTIVE 검증
2. mapping 조회
3. equipment 최신 상태 재조회
4. status ∈ {CHECKED_OUT, OVERDUE} 검증
5. equipments.status = AVAILABLE
6. currentHolderUserId/name 제거
7. lastCheckinAt = serverTimestamp
8. equipment_logs CHECK_IN 생성
9. audit_logs 필요 시 생성

---

forceUpdateEquipmentStatus(payload)

목적

관리자 예외 복구

입력

- equipmentId
- nextStatus
- reason
- note

처리

1. ADMIN 이상 검증
2. nextStatus 허용값 검증
3. before snapshot 보관
4. 상태 강제 변경
5. currentHolder 정리 여부 정책 적용
6. equipment_logs FORCE_UPDATE 생성
7. audit_logs 생성

---

markEquipmentOverdueBatch()

목적

미반입 자동 판정 배치

실행 방식

- Cloud Scheduler 주기 실행

처리

1. CHECKED_OUT 장비 조회
2. lastCheckoutAt 기준 초과 여부 판단
3. OVERDUE 전환
4. 필요 시 equipment_logs 생성
5. 대시보드/목록용 상태 명확화

3-4. 함수 목록 — 로그/운영 도메인

appendAuditLog(payload)

직접 호출형보다 내부 helper로 사용하는 것을 권장한다.

createDownloadLog(payload)

엑셀 다운로드 시 관리자 액션 기록.

syncSchedulePhotoFlags(scheduleId)

schedule_photos 집계 후: - schedules.hasBeforePhotos - schedules.hasAfterPhotos 갱신

recomputeScheduleStatus(scheduleId)

일정 상태 집계 재계산의 핵심 함수. 여러 함수가 마지막에 공통 호출한다.

3-5. 공통 함수 설계 규칙

모든 callable 함수 공통

- request.auth 필수
- users/{uid}.isActive 확인
- role 기반 분기
- schema validation 수행
- business error와 system error 구분
- 항상 서버 timestamp 사용

모든 트랜잭션 함수 공통

- 최신 문서 재조회
- 클라이언트 전달 상태값 신뢰 금지
- 성공 시 로그 생성
- 실패 시 부분 커밋 금지

모든 감사 로그 공통

- actorUserId
- actorUserName
- actorRole

- action
- reason
- beforeData
- afterData
- createdAt

---

4. 실제 프로젝트 코드 구조 — React + Firebase 바로 구현

이 장의 목적은 지금 바로 개발을 시작할 수 있는 폴더 구조와 모듈 경계를 고정하는 것이다.

핵심 원칙: - 라우트와 도메인을 함께 본다. - UI, 상태, 서비스, 타입을 분리한다. - 관리자와 사원 영역을 분리하되 공통 컴포넌트는 재사용한다. - Firebase SDK 직접 호출은 service/repository 층으로 모은다.

4-1. 권장 기술 스택

- React 18
- Vite
- TypeScript
- Tailwind CSS
- React Router
- Zustand
- TanStack Query
- Firebase Auth
- Firestore
- Storage
- Functions

4-2. 최상위 폴더 구조

```
hella-ops/
├─ .env.local
├─ .env.production
├─ firebase.json
├─ firestore.rules
├─ firestore.indexes.json
├─ storage.rules
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
├─ functions/
│  ├─ src/
│  └─ package.json
```

```
├─ public/
│  └─ logos/
├─ docs/
│  ├─ 01_mvp_spec.md
│  ├─ 02_menu_function_specs.md
│  ├─ 03_project_roadmap.md
│  ├─ 04_screen_map.md
│  ├─ 05_database_schema.md
│  ├─ 06_permission_policy.md
│  ├─ 07_firestore_rules.md
│  ├─ 08_nfc_engine.md
│  └─ 09_api_contract.md
└─ src/
   ├─ app/
   ├─ pages/
   ├─ features/
   ├─ components/
   ├─ layouts/
   ├─ services/
   ├─ repositories/
   ├─ stores/
   ├─ hooks/
   ├─ lib/
   ├─ types/
   ├─ constants/
   ├─ routes/
   ├─ utils/
   └─ main.tsx
```

4-3. src/app 구조

```
src/app/
├─ App.tsx
├─ providers.tsx
├─ router.tsx
├─ auth-bootstrap.ts
└─ query-client.ts
```

역할: - 전역 Provider 주입 - Router 초기화 - Auth 세션 부트스트랩

4-4. src/routes 구조

```
src/routes/
├─ guards/
```

```
│   ├─ RequireAuth.tsx
│   ├─ RequireAdmin.tsx
│   ├─ RequireEmployee.tsx
│   └─ RequireActiveUser.tsx
├─ AppRoutes.tsx
└─ routePaths.ts
```

핵심: - /login - /app/ 관리자 - /m/ 사원

4-5. src/layouts 구조

```
src/layouts/
├─ AdminLayout.tsx
├─ EmployeeLayout.tsx
└─ AuthLayout.tsx
```

AdminLayout

- Sidebar
- Header
- Breadcrumb
- Content

EmployeeLayout

- MobileHeader
- Content
- BottomTabBar

4-6. src/pages 구조

```
src/pages/
├─ auth/
│   └─ LoginPage.tsx
├─ admin/
│   ├─ dashboard/
│   │   └─ AdminDashboardPage.tsx
│   ├─ schedules/
│   │   ├─ AdminScheduleListPage.tsx
│   │   ├─ AdminScheduleFormPage.tsx
│   │   └─ AdminScheduleDetailPage.tsx
│   ├─ equipments/
│   │   ├─ AdminEquipmentListPage.tsx
│   │   ├─ AdminEquipmentFormPage.tsx
│   │   └─ AdminEquipmentDetailPage.tsx
```

```
│   ├─ users/
│   ├─ manuals/
│   ├─ business/
│   ├─ closings/
│   ├─ regular-cleaning/
│   └─ settings/
└─ employee/
    ├─ home/
    │   └─ EmployeeHomePage.tsx
    ├─ schedules/
    │   └─ EmployeeSchedulePage.tsx
    ├─ closing/
    │   └─ EmployeeClosingPage.tsx
    ├─ equipments/
    │   ├─ EmployeeEquipmentOutPage.tsx
    │   └─ EmployeeEquipmentInPage.tsx
    ├─ manuals/
    └─ profile/
```

4-7. src/features 구조

도메인 중심으로 기능을 묶는다.

```
src/features/
├─ auth/
│   ├─ components/
│   ├─ hooks/
│   ├─ services/
│   └─ types.ts
├─ schedules/
│   ├─ components/
│   ├─ hooks/
│   ├─ api/
│   ├─ model/
│   ├─ validators/
│   └─ types.ts
├─ closings/
├─ equipments/
├─ users/
├─ manuals/
├─ business/
├─ dashboard/
└─ regular-cleaning/
```

장점: - 도메인 캡슐화 - 화면과 로직 재사용 용이 - 관리자/사원 화면이 같은 도메인 로직 공유 가능

4-8. src/components 공통 컴포넌트

```
src/components/
├─ ui/
│  ├─ Button.tsx
│  ├─ Input.tsx
│  ├─ Select.tsx
│  ├─ Modal.tsx
│  ├─ Drawer.tsx
│  ├─ BottomSheet.tsx
│  ├─ Badge.tsx
│  ├─ Tabs.tsx
│  ├─ Toast.tsx
│  └─ Skeleton.tsx
├─ forms/
│  ├─ TimePickerField.tsx
│  ├─ AddressField.tsx
│  ├─ WorkerMultiSelect.tsx
│  └─ PhotoUploader.tsx
├─ calendar/
├─ nfc/
│  └─ NfcScanButton.tsx
└─ feedback/
```

4-9. src/services / repositories 분리

services

비즈니스 오케스트레이션 층

```
src/services/
├─ authService.ts
├─ scheduleService.ts
├─ closingService.ts
├─ equipmentService.ts
├─ dashboardService.ts
└─ manualService.ts
```

repositories

Firebase SDK 직접 접근 층

```
src/repositories/
├─ authRepository.ts
├─ usersRepository.ts
```

```
├─ schedulesRepository.ts
├─ scheduleWorkersRepository.ts
├─ scheduleClosingsRepository.ts
├─ schedulePhotosRepository.ts
├─ equipmentsRepository.ts
├─ equipmentLogsRepository.ts
├─ manualsRepository.ts
└─ settingsRepository.ts
```

원칙: - page/component 에서 firestore 직접 호출 금지 - repository는 CRUD/조회 담당 - service는 도메인 규칙/조합 담당

4-10. src/stores 구조

```
src/stores/
├─ authStore.ts
├─ uiStore.ts
├─ employeeScheduleUiStore.ts
├─ employeeClosingUiStore.ts
├─ adminScheduleFilterStore.ts
└─ equipmentScanStore.ts
```

상태관리 분리 원칙

- 서버 데이터: TanStack Query
- UI 로컬 상태: Zustand
- 폼 상태: 페이지 내부 state 또는 react-hook-form 도입 가능

4-11. src/types 구조

```
src/types/
├─ auth.ts
├─ user.ts
├─ schedule.ts
├─ closing.ts
├─ photo.ts
├─ equipment.ts
├─ manual.ts
├─ dashboard.ts
└─ common.ts
```

필수 enum 예시: - UserRole - ScheduleStatus - WorkStatus - ClosingStatus - EquipmentStatus - EquipmentActionType

---

4-12. Firebase functions 구조

```
functions/src/
├─ index.ts
├─ config/
│  └─ firebase.ts
├─ common/
│  ├─ auth.ts
│  ├─ errors.ts
│  ├─ validators.ts
│  ├─ timestamps.ts
│  └─ audit.ts
├─ modules/
│  ├─ schedules/
│  │  ├─ createScheduleWithWorkers.ts
│  │  ├─ updateScheduleAndWorkers.ts
│  │  ├─ cancelSchedule.ts
│  │  ├─ recomputeScheduleStatus.ts
│  │  ├─ recordWorkStart.ts
│  │  ├─ recordWorkEnd.ts
│  │  ├─ submitClosing.ts
│  │  └─ approveClosing.ts
│  ├─ equipments/
│  │  ├─ assignNfcTag.ts
│  │  ├─ processNfcScan.ts
│  │  ├─ checkoutEquipment.ts
│  │  ├─ checkinEquipment.ts
│  │  ├─ forceUpdateEquipmentStatus.ts
│  │  └─ markEquipmentOverdueBatch.ts
│  ├─ dashboards/
│  │  └─ getDashboardSummary.ts
│  └─ users/
│     └─ createEmployeeUser.ts
└─ triggers/
   ├─ onSchedulePhotoWrite.ts
   ├─ onScheduleClosingWrite.ts
   └─ onUserStatusChange.ts
```

4-13. 페이지별 구현 연결 표

로그인

- page: LoginPage.tsx
- service: authService.ts
- repository: authRepository.ts / usersRepository.ts
- route: /login

사원 홈

- page: EmployeeHomePage.tsx
- service: dashboardService.ts
- query: getTodaySchedules(userId), getEquipmentStatus(userId)
- route: /m/home

사원 일정

- page: EmployeeSchedulePage.tsx
- service: scheduleService.ts
- query: getEmployeeSchedulesByScope()
- route: /m/calendar

사원 마감

- page: EmployeeClosingPage.tsx
- service: closingService.ts
- functions: recordWorkStart, recordWorkEnd, submitClosing
- route: /m/closing/:scheduleId

관리자 대시보드

- page: AdminDashboardPage.tsx
- service: dashboardService.ts
- widgets: KPI, Alert, QuickAction, Charts
- route: /app/dashboard

관리자 일정관리

- pages:
- AdminScheduleListPage
- AdminScheduleFormPage
- AdminScheduleDetailPage
- service: scheduleService.ts
- functions:
- createScheduleWithWorkers
- updateScheduleAndWorkers
- cancelSchedule
- routes:
- /app/schedules
- /app/schedules/new
- /app/schedules/:id

관리자 장비관리

- pages:
- AdminEquipmentListPage
- AdminEquipmentFormPage
- AdminEquipmentDetailPage
- service: equipmentService.ts
- functions:
- assignNfcTag

- forceUpdateEquipmentStatus
- routes:
- /app/equipments
- /app/equipments/new
- /app/equipments/:id

4-14. Query key 구조 권장안

```
['auth', uid]
['dashboard', role, range]
['employeeSchedules', uid, scope, date]
['employeeCalendarMarks', uid, month]
['scheduleDetail', scheduleId]
['scheduleWorkers', scheduleId]
['scheduleClosing', scheduleId, uid]
['schedulePhotos', scheduleId]
['adminSchedules', filters]
['equipmentList', filters]
['equipmentDetail', equipmentId]
['equipmentLogs', filters]
```

4-15. UI/로직 분리 원칙 — 실제 적용

page 역할

- 데이터 요청 조합
- 화면 조립
- 라우팅 처리

feature component 역할

- 화면 단위 UI 구현

service 역할

- 호출 순서, 정책 조합
- functions / repository 호출 orchestration

repository 역할

- Firebase SDK 단일 책임

validator 역할

- 클라이언트 선검증

functions 역할

- 최종 검증 + 상태 확정 + 로그 생성

4-16. 개발 시작 우선순위

1차 필수

1. 로그인/권한 분기
2. 공통 레이아웃
3. 관리자 대시보드 기초 KPI
4. 관리자 일정관리 CRUD + 배정
5. 사원 홈
6. 사원 일정
7. 사원 시작/종료/마감
8. 관리자 장비 목록
9. 사원 장비 반출/반입
10. Rules/Functions 연결

2차 강화

1. 일정 상세 검토 탭 강화
2. 관리자 장비 상세 + 로그
3. 정기청소 관리
4. 다운로드 로그
5. OVERDUE 배치 자동화
6. Dashboard 차트/최근활동 고도화

4-17. 실제 구현 금지사항

1. page 컴포넌트에서 firestore 직접 write 금지
2. equipment 상태를 클라이언트에서 직접 set 금지
3. schedule.status 를 클라이언트에서 직접 set 금지
4. audit_logs 직접 기록 금지
5. Rules 없이 관리자 화면부터 개발 금지
6. worker 배정 변경을 schedule_workers 무시한 채 schedules.workerIds만 수정 금지

4-18. 최종 결론

이 프로젝트의 개발 직전 핵심은 4가지다.

첫째, **필드 단위 화이트리스트로 권한을 세분화해야 한다.** 둘째, **상태 전이는 허용된 흐름만 서버에서 확정해야 한다.** 셋째, **NFC/장비/마감/상태 변경은 Functions 트랜잭션으로 묶어야 한다.** 넷째, **React 프론트는 화면, 서비스, 저장소, 타입을 분리한 구조로 바로 시작해야 한다.**

이 문서 기준으로 다음 단계는 실제 개발용 산출물이다.

- firestore.rules 최종 코드
- storage.rules 최종 코드
- functions/src 초기 골격
- src 폴더 초기 코드 골격
- 라우터/권한가드/타입 정의

이 문서는 그 4가지를 만들기 위한 최종 실행 캔버스다.